

Assignments 01 :-Create an image classifier using a traditional machine learning approach Random Forest and analyze how minor modifications (like noise) affect its predictions. (dataset: MNIST (handwritten digits))

Name:- Vaishnavi Gavali

ROLL NO:-EN24107044

Div:-AI&DS-A(C)

```
In [ ]: pip install numpy pandas matplotlib seaborn scikit-learn
```

```
In [3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
In [4]: train_data = pd.read_csv("mnist_train.csv")
test_data = pd.read_csv("mnist_test.csv")

print("Train Shape:", train_data.shape)
print("Test Shape:", test_data.shape)
```

Train Shape: (60000, 785)

Test Shape: (10000, 785)

```
In [5]: train_data.head()
```

```
Out[5]:
```

	label	1x1	1x2	1x3	1x4	1x5	1x6	1x7	1x8	1x9	...	28x19	28x20	28x21	28x22	28x23	28x24
0	5	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
2	4	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
3	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
4	9	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0

5 rows × 785 columns

```
In [6]: test_data.head()
```

Out[6]:

	label	1x1	1x2	1x3	1x4	1x5	1x6	1x7	1x8	1x9	...	28x19	28x20	28x21	28x22	28x23	28x24
0	7	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
1	2	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
4	4	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0

5 rows × 785 columns

```
In [7]: X_train = train_data.drop("label", axis=1)
        y_train = train_data["label"]

        X_test = test_data.drop("label", axis=1)
        y_test = test_data["label"]

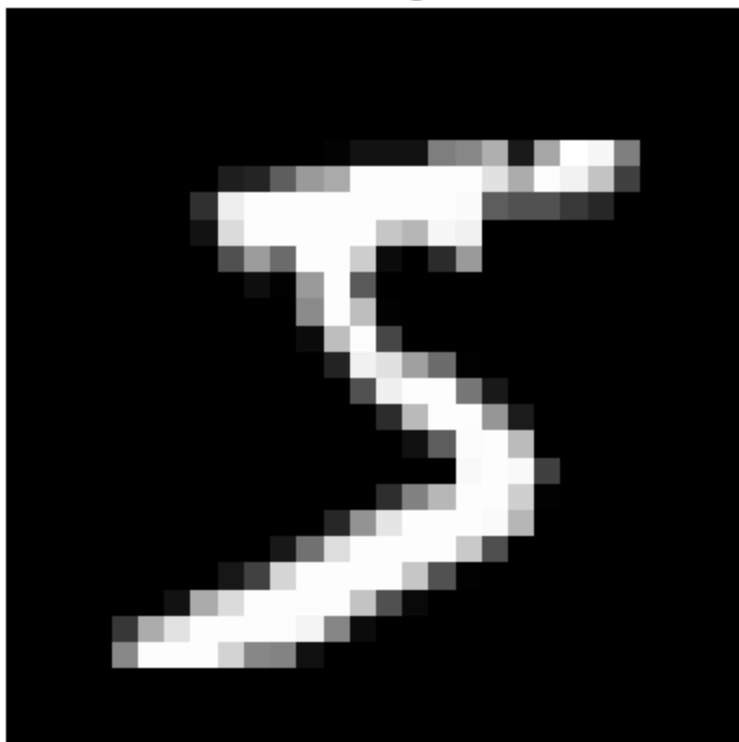
        print(X_train.shape)
        print(y_train.shape)
        print(X_test.shape)
        print(y_test.shape)

        (60000, 784)
        (60000,)
        (10000, 784)
        (10000,)
```

```
In [8]: plt.imshow(
        X_train.iloc[0].values.reshape(28, 28),
        cmap="gray"
    )

    plt.title("MNIST Digit: " + str(y_train.iloc[0]))
    plt.axis("off")
    plt.show()
```

MNIST Digit: 5



```
In [9]: model = RandomForestClassifier(  
        n_estimators=100,  
        random_state=42,  
        n_jobs=-1  
    )
```

```
In [10]: print("Training started...")  
  
model.fit(X_train, y_train)  
  
print("Training completed!")
```

```
Training started...  
Training completed!
```

```
In [11]: y_pred = model.predict(X_test)
```

```
In [12]: accuracy = accuracy_score(y_test, y_pred)  
  
print("Accuracy =", accuracy * 100, "%")
```

```
Accuracy = 97.05 %
```

```
In [13]: def add_noise(images, noise_level):  
  
        noise = np.random.normal(  
            0,  
            noise_level,  
            images.shape  
        )  
  
        noisy_images = images + noise
```

```

noisy_images = np.clip(
    noisy_images,
    0,
    255
)

return noisy_images

```

```

In [14]: noise_levels = [0, 20, 40, 60, 80, 100]

accuracies = []

for level in noise_levels:

    if level == 0:
        X_noisy = X_test
    else:
        X_noisy = pd.DataFrame(
            add_noise(X_test.values, level),
            columns=X_test.columns
        )

    prediction = model.predict(X_noisy)

    acc = accuracy_score(y_test, prediction)

    accuracies.append(acc * 100)

    print(
        "Noise Level:", level,
        "Accuracy:", round(acc * 100, 2), "%"
    )

```

```

Noise Level: 0 Accuracy: 97.05 %
Noise Level: 20 Accuracy: 73.46 %
Noise Level: 40 Accuracy: 65.21 %
Noise Level: 60 Accuracy: 60.45 %
Noise Level: 80 Accuracy: 56.54 %
Noise Level: 100 Accuracy: 52.62 %

```

```

In [15]: plt.figure(figsize=(8,5))

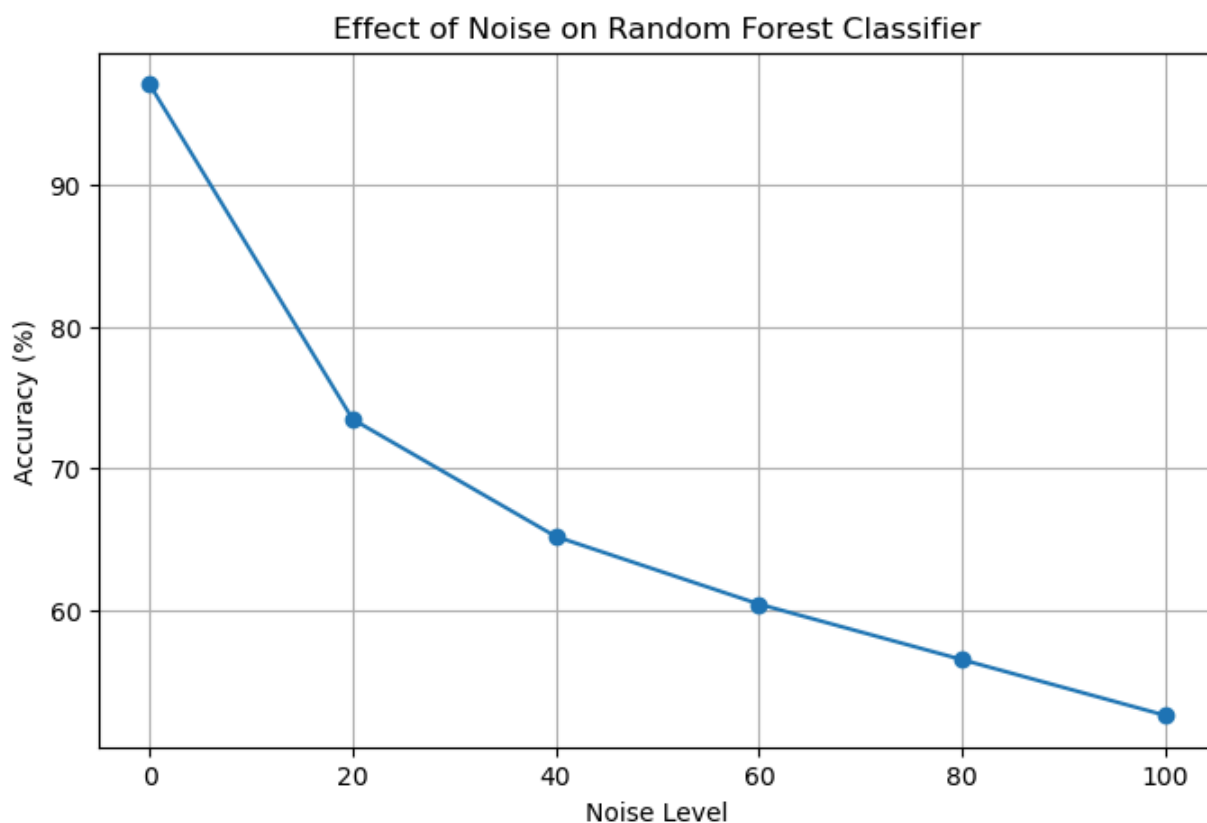
plt.plot(
    noise_levels,
    accuracies,
    marker="o"
)

plt.xlabel("Noise Level")
plt.ylabel("Accuracy (%)")

plt.title(
    "Effect of Noise on Random Forest Classifier"
)

plt.grid()
plt.show()

```



```
In [17]: accuracy_percent = np.array(accuracies) * 100
accuracy_loss = accuracy_percent[0] - accuracy_percent

plt.figure(figsize=(10, 6))

plt.plot(
    noise_levels,
    accuracy_loss,
    marker="o",
    markersize=8,
    linewidth=2.5,
    color="red"
)

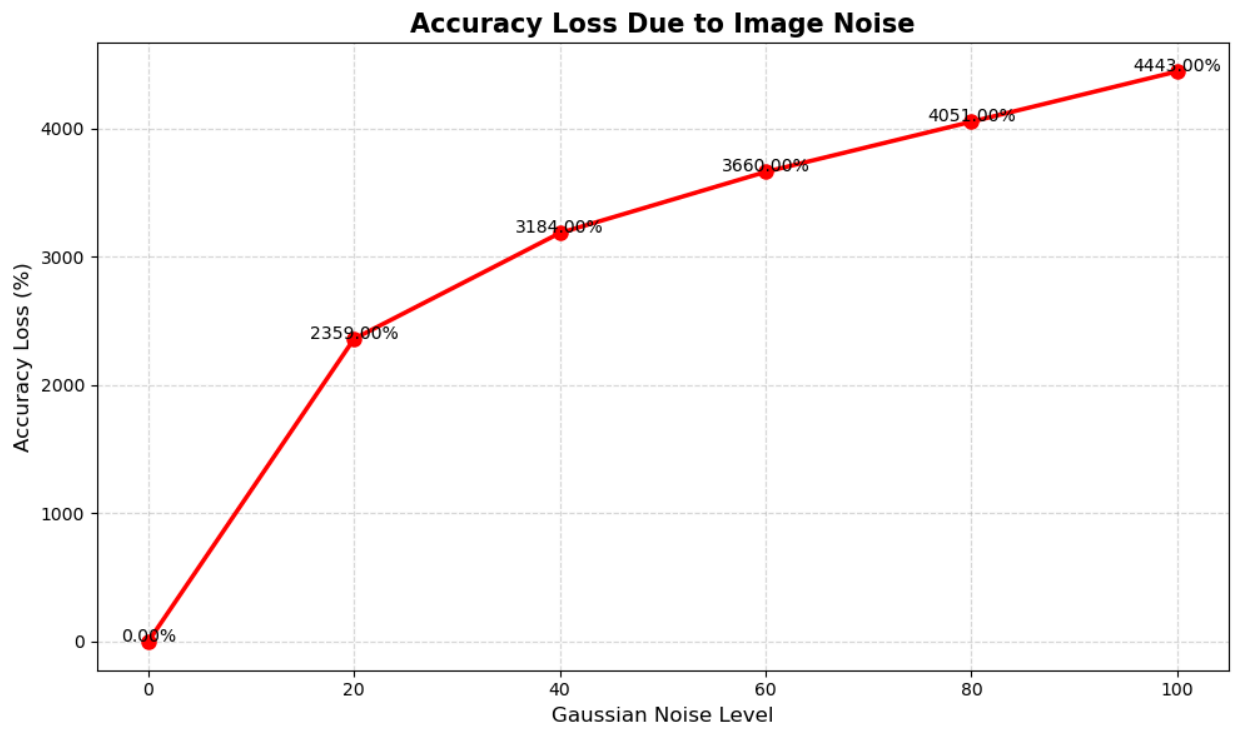
for x, y in zip(noise_levels, accuracy_loss):
    plt.text(
        x,
        y + 0.5,
        f"{y:.2f}%",
        ha="center",
        fontsize=10
    )

plt.xlabel("Gaussian Noise Level", fontsize=12)
plt.ylabel("Accuracy Loss (%)", fontsize=12)

plt.title(
    "Accuracy Loss Due to Image Noise",
    fontsize=15,
    fontweight="bold"
)

plt.grid()
```

```
True,  
linestyle="--",  
alpha=0.5  
)  
  
plt.tight_layout()  
plt.show()
```



In []: